A to I

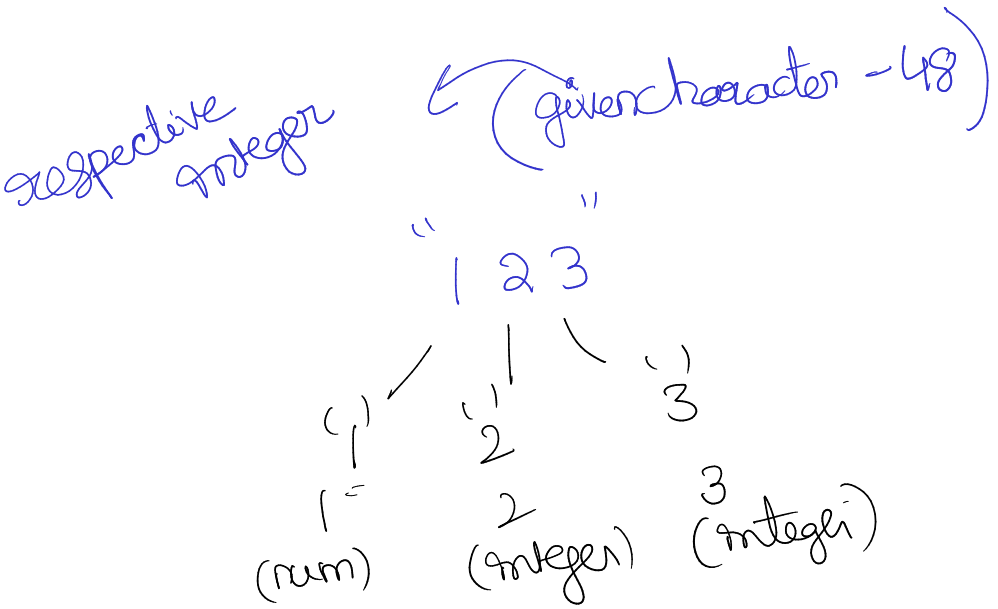
i/p → "123" o/p → 123 (integer)

Valid i/p

"123"
"357"
"+123"
"-123"

Invalid i/p

"12*3"
"1A23"
"12+3"




```
i=0;
while(s[i])
{ if((s[0]=='+' || s[0]=='-'))
  { if(s[0]=='-')
      f=1;
    i++;
  }
if(s[i]>='0' && s[i]<='9')
  num = (num*10) + (s[i]-48)
```

&& i==0

```
else
{ print ("Invalid")
    return 0;
}
i++;
}
```

"+123"
"-123"
n = -123
(1) n*(-1)

```
if (f==1)
num = num*(-1);
printf ("%d", num);
```

AtoF

i/p → "123.325" o/p ⇒ 123.325 (float number)

"123325" ⇒ 123325 * 0.001
= 123.325

i/p → 123.23.35
error

```
else if ( s[i]=='.')
{   dot=1;
    i++;  → cnt++;
else
{ pf("Invalid");
```

cnt > 1
pf(Inval
return

```
if (dot == 1)
    dec = dec * 0.1;
```

123 X 356

dot = 1

dec = 10

}

condn +ve −ve

n * dec

n * 0.001

123.356 12.356

i/p → 123 (integer) o/p → "123"

I to A

i/p

i/p → 1234

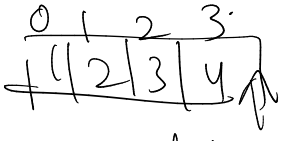
o/p → "1234"

A to I

cnt (count no. of digits)

4

123



#defn n 24
main

declare a char c[20]

char c[n];

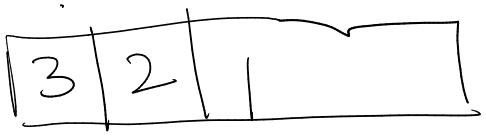
strlen = 4 ⇒ n = 4

for (i = n-1; i >= 0; i--)

s[i] = conversion ant to char

⇒

123



reverse the array

1 2 3

F to A

i/p → 123.356 (float) o/p → "123.356"

Seperate integral decimal part

123 + 0.356

i = 123

.356

f = 123.356000

i f1

i = f;

f1 = f - i

0.356

(356000)

356

0.356 × 1000000

1.56

653

1. run a loop checking each character & next character vowel or not till reaches NULL.

2. To check 2 characters are vowels are not use a fun vowel_check to confirm it

3. If the 2nd step is true hide those 2 vowels

4. finally print the o/p string.

```
char s[50]; int i;
gets(s)
puts(s)
for(i=0; s[i]; i++)
{
    if ( check_vowel(s[i])
         && check_vowel(s[i+1])
    {
        s[i] = '*';
        s[i+1] = '*';
    }
}
printf("%s", s);
```

```
int check_vowel(char a)
{
    char V[10]={'a','e','i','o','u',
                'A','E','I','O','U'};
    for(i=0;i<10;i++)
    {
        if(V[i]==a)
            return 1;
    }
    return 0;
}
```